

Building A Linear Reservoir From Scratch

Today we will build a simple hydrological model.

The model is called a **linear reservoir**.

It has one storage:

`storage_mm`

It has one parameter:

`residence_time_days`

The model says:

`outflow = storage / residence time`

`new storage = old storage + inflow - outflow`

We will start with one day, then many days, then a reusable model function.

Построение линейного резервуара с нуля

Сегодня мы шаг за шагом построим простую гидрологическую модель.

Эта модель называется **линейный резервуар**.

У нее есть одно хранилище:

`storage_mm`

И один параметр:

`residence_time_days`

Модель задается так:

`outflow = storage / residence time`

`new storage = old storage + inflow - outflow`

Мы начнем с одного дня, затем перейдем к нескольким дням, а потом оформим модель как функцию, которую можно использовать повторно.

Python Environment

In VS Code, open a terminal.

On Windows PowerShell:

```
py -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

```
python -m pip install ipykernel numpy matplotlib pandas
```

On macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install ipykernel numpy matplotlib pandas
```

Then select the `.venv` kernel in this notebook.

Среда Python

В VS Code откройте терминал.

В Windows PowerShell:

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install ipykernel numpy matplotlib pandas
```

В macOS или Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install ipykernel numpy matplotlib pandas
```

Затем выберите ядро `.venv` для этого ноутбука.

1. One Reservoir Timestep By Hand

Let us begin with one day.

The reservoir starts with `50 mm` of water in storage.

The residence time is `10 days`.

Today, `4 mm` of water enters the reservoir.

We use `storage_before_mm` and `storage_after_mm` so it is clear which storage belongs to which moment in time.

1. Один шаг резервуара вручную

Начнем с одного дня.

В начале в резервуаре хранится `50 mm` воды.

Время пребывания воды равно `10 days`.

Сегодня в резервуар поступает `4 mm` воды.

Мы используем `storage_before_mm` и `storage_after_mm`, чтобы было ясно, к какому моменту времени относится хранилище.

```
In [1]: storage_before_mm = 50.0
        residence_time_days = 10.0
        inflow_mm = 4.0

        outflow_mm = storage_before_mm / residence_time_days
        storage_after_mm = storage_before_mm + inflow_mm - outflow_mm

        print("outflow:", outflow_mm, "mm/day")
        print("storage after:", storage_after_mm, "mm")
```

```
outflow: 5.0 mm/day
storage after: 49.0 mm
```

This is one model timestep.

The storage before the step was `50 mm`.

Because `residence_time_days = 10`, one tenth of the storage leaves in one day.

The inflow was `4 mm`.

So the reservoir loses `5 mm`, gains `4 mm`, and ends with `49 mm`.

Это один временной шаг модели.

Хранилище до шага было равно `50 mm`.

Поскольку `residence_time_days = 10`, за один день из резервуара выходит одна десятая хранилища.

Приток был равен `4 mm`.

Поэтому резервуар теряет `5 mm`, получает `4 mm` и заканчивает день с `49 mm`.

2. A Second Day Creates The Idea Of State

Now we simulate a second day.

The storage after day 1 becomes the storage before day 2.

2. Второй день вводит идею состояния

Теперь смоделируем второй день.

Хранилище после дня 1 становится хранилищем до дня 2.

```
In [2]: storage_before_mm = storage_after_mm
inflow_mm = 0.0

outflow_mm = storage_before_mm / residence_time_days
storage_after_mm = storage_before_mm + inflow_mm - outflow_mm

print("outflow:", outflow_mm, "mm/day")
print("storage after:", storage_after_mm, "mm")
```

outflow: 4.9 mm/day
storage after: 44.1 mm

The model has memory.

The result from day 1 becomes the starting point for day 2.

That changing memory is called the **state** of the model.

У модели есть память.

Результат дня 1 становится начальной точкой для дня 2.

Эта изменяющаяся память называется **состоянием** модели.

3. Many Days Create The Need For Lists And Loops

Copying the same equations once per day would be painful.

Instead, we store several days of inflow in a list.

3. Несколько дней создают потребность в списках и циклах

Копировать одни и те же уравнения для каждого дня было бы неудобно.

Вместо этого мы сохраним приток за несколько дней в список.

```
In [3]: inflow_series_mm = [4.0, 0.0, 12.0, 2.0, 0.0]

print(inflow_series_mm)
```

[4.0, 0.0, 12.0, 2.0, 0.0]

```
In [4]: storage_before_mm = 50.0
residence_time_days = 10.0

for inflow_mm in inflow_series_mm:
    outflow_mm = storage_before_mm / residence_time_days
    storage_after_mm = storage_before_mm + inflow_mm - outflow_mm

    print("inflow:", inflow_mm, "outflow:", outflow_mm, "storage after:", st
```

```
storage_before_mm = storage_after_mm
```

```
inflow: 4.0 outflow: 5.0 storage after: 49.0
inflow: 0.0 outflow: 4.9 storage after: 44.1
inflow: 12.0 outflow: 4.41 storage after: 51.69
inflow: 2.0 outflow: 5.169 storage after: 48.521
inflow: 0.0 outflow: 4.8521 storage after: 43.6689
```

The loop repeats the same model step for each inflow value.

At the end of each day, we update `storage_before_mm`.

That updated storage becomes the starting storage for the next day.

Цикл повторяет один и тот же шаг модели для каждого значения притока.

В конце каждого дня мы обновляем `storage_before_mm`.

Это обновленное хранилище становится начальным хранилищем для следующего дня.

4. Store The Simulation Output

Printing is useful while we are learning, but it does not give us a result we can plot or analyze.

So we store the simulated storage and outflow in lists.

4. Сохраняем результат симуляции

Печать значений полезна, когда мы учимся, но она не дает нам результата, который удобно строить на графике или анализировать.

Поэтому мы сохраняем смоделированные хранилище и отток в списки.

```
In [5]: storage_before_mm = 50.0
        residence_time_days = 10.0

        storage_series_mm = []
        outflow_series_mm = []

        for inflow_mm in inflow_series_mm:
            outflow_mm = storage_before_mm / residence_time_days
            storage_after_mm = storage_before_mm + inflow_mm - outflow_mm

            storage_series_mm.append(storage_after_mm)
            outflow_series_mm.append(outflow_mm)

            storage_before_mm = storage_after_mm
```

```
print("storage:", storage_series_mm)
print("outflow:", outflow_series_mm)
```

```
storage: [49.0, 44.1, 51.69, 48.521, 43.6689]
outflow: [5.0, 4.9, 4.41, 5.169, 4.8521]
```

Now we have a simulated time series.

This is the first real model output.

Теперь у нас есть смоделированный временной ряд.

Это первый настоящий результат модели.

5. Plot The Toy Simulation

A plot helps us see how storage and outflow change through time.

5. Строим график простой симуляции

График помогает увидеть, как хранилище и отток меняются во времени.

```
In [6]: import matplotlib.pyplot as plt

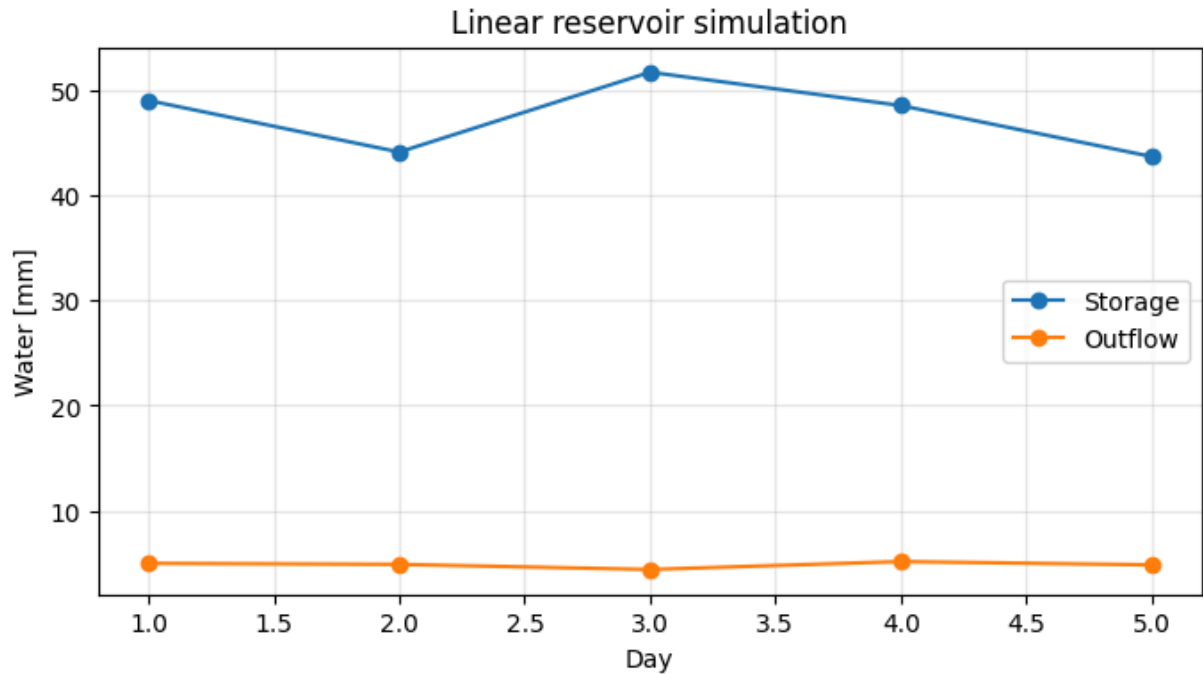
days = [1, 2, 3, 4, 5]

fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(days, storage_series_mm, marker="o", label="Storage")
ax.plot(days, outflow_series_mm, marker="o", label="Outflow")

ax.set_xlabel("Day")
ax.set_ylabel("Water [mm]")
ax.set_title("Linear reservoir simulation")
ax.grid(alpha=0.3)
ax.legend()

plt.show()
```



6. One Timestep Becomes A Function

The loop works, but the model equations are hidden inside it.

The hydrological idea has a name:

one timestep of the reservoir

So we put one timestep into a function.

6. Один временной шаг становится функцией

Цикл работает, но уравнения модели спрятаны внутри него.

У этой гидрологической идеи есть название:

one timestep of the reservoir

Поэтому мы помещаем один временной шаг в функцию.

```
In [7]: def step(
    storage_before_mm: float,
    residence_time_days: float,
    inflow_mm: float,
) -> tuple[float, float]:
    outflow_mm = storage_before_mm / residence_time_days
    storage_after_mm = storage_before_mm + inflow_mm - outflow_mm
    return storage_after_mm, outflow_mm
```

```
In [8]: storage_after_mm, outflow_mm = step(
    storage_before_mm=50.0,
    residence_time_days=10.0,
```

```

    inflow_mm=4.0,
)

print(storage_after_mm)
print(outflow_mm)

```

49.0

5.0

The function separates the model equation from the time loop.

`step()` answers one question: given the storage before the timestep, the residence time, and the inflow, what is the storage after the timestep and today's outflow?

Функция отделяет уравнение модели от цикла по времени.

`step()` отвечает на один вопрос: если известны хранилище до временного шага, время пребывания и приток, каким будет хранилище после временного шага и сегодняшний отток?

7. Many Timesteps Become A Run Function

Now we write one function for the full simulation.

The `step()` function simulates one day.

The `run_linear_reservoir()` function simulates a full time series.

7. Несколько временных шагов становятся функцией запуска модели

Теперь мы напишем одну функцию для всей симуляции.

Функция `step()` моделирует один день.

Функция `run_linear_reservoir()` моделирует полный временной ряд.

```

In [9]: def run_linear_reservoir(
        initial_storage_mm: float,
        residence_time_days: float,
        inflow_series_mm: list[float],
    ) -> tuple[list[float], list[float]]:
        storage_before_mm = initial_storage_mm

        storage_series_mm = []
        outflow_series_mm = []

        for inflow_mm in inflow_series_mm:
            storage_after_mm, outflow_mm = step(
                storage_before_mm=inflow_mm,

```



```

        residence_time_days=residence_time_days,
        inflow_mm=inflow_mm,
    )

    storage_series_mm.append(storage_after_mm)
    outflow_series_mm.append(outflow_mm)

    storage_before_mm = storage_after_mm

    return storage_series_mm, outflow_series_mm

```

```

In [10]: storage_series_mm, outflow_series_mm = run_linear_reservoir(
        initial_storage_mm=50.0,
        residence_time_days=10.0,
        inflow_series_mm=inflow_series_mm,
    )

    print(storage_series_mm)
    print(outflow_series_mm)

```

```

[49.0, 44.1, 51.69, 48.521, 43.6689]
[5.0, 4.9, 4.41, 5.169, 4.8521]

```

Now the model has two levels:

- `step()` simulates one day
- `run_linear_reservoir()` simulates a full time series

This is a common structure in hydrological modelling.

Теперь у модели есть два уровня:

- `step()` моделирует один день
- `run_linear_reservoir()` моделирует полный временной ряд

Это распространенная структура в гидрологическом моделировании.

8. Intermezzo: NumPy Arrays And Positions

Earlier, we stored results with `.append()`.

That worked because Python lists can grow:

```
outflow_series_mm.append(outflow_mm)
```

For longer numerical simulations, we often use NumPy arrays.

With NumPy, we usually create the full output array first.

Then we fill one position at a time.

To do that, we need two things during the loop:

- the position number
- the value at that position

`enumerate(...)` gives us both.

8. Интермеццо: массивы NumPy и позиции

Раньше мы сохраняли результаты с помощью `.append()`.

Это работало, потому что списки Python могут расти:

```
outflow_series_mm.append(outflow_mm)
```

Для более длинных численных симуляций мы часто используем массивы NumPy.

В NumPy обычно сначала создают весь выходной массив.

Затем мы заполняем по одной позиции за раз.

Для этого в цикле нам нужны две вещи:

- номер позиции
- значение в этой позиции

`enumerate(...)` дает нам и то, и другое.

```
In [11]: import numpy as np

example_inflow = np.array([4.0, 0.0, 12.0])

for i, inflow_mm in enumerate(example_inflow):
    print("position:", i, "value:", inflow_mm)
```

```
position: 0 value: 4.0
position: 1 value: 0.0
position: 2 value: 12.0
```

```
In [12]: example_outflow = np.zeros(len(example_inflow))

for i, inflow_mm in enumerate(example_inflow):
    example_outflow[i] = inflow_mm * 0.5

print(example_outflow)
```

```
[2. 0. 6.]
```

```
In [13]: def run_linear_reservoir(
    initial_storage_mm: float,
    residence_time_days: float,
    inflow_series_mm: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    storage_series_mm = np.zeros(len(inflow_series_mm))
    outflow_series_mm = np.zeros(len(inflow_series_mm))
```

```

storage_before_mm = initial_storage_mm

for i, inflow_mm in enumerate(inflow_series_mm):
    storage_after_mm, outflow_mm = step(
        storage_before_mm=storage_before_mm,
        residence_time_days=residence_time_days,
        inflow_mm=inflow_mm,
    )

    storage_series_mm[i] = storage_after_mm
    outflow_series_mm[i] = outflow_mm

    storage_before_mm = storage_after_mm

return storage_series_mm, outflow_series_mm

```

9. Main Experiment: Pulse Response

Now we do a controlled experiment.

We put **100 mm** of water into the reservoir on the first day.

Then we watch how the reservoir drains.

This helps us understand what **residence_time_days** controls.

9. Главный эксперимент: отклик на импульс

Теперь проведем контролируемый эксперимент.

В первый день мы помещаем в резервуар **100 mm** воды.

Затем наблюдаем, как резервуар опорожняется.

Это помогает понять, чем управляет **residence_time_days**.

```

In [14]: n_days = 60

inflow_series_mm = np.zeros(n_days)
inflow_series_mm[0] = 100.0

days = np.arange(n_days)

print(inflow_series_mm[:10])

[100.  0.  0.  0.  0.  0.  0.  0.  0.  0.]

```

```

In [15]: storage_mm, outflow_mm = run_linear_reservoir(
    initial_storage_mm=0.0,
    residence_time_days=10.0,
    inflow_series_mm=inflow_series_mm,
)

```

```
print("peak outflow:", outflow_mm.max())
print("final storage:", storage_mm[-1])
```

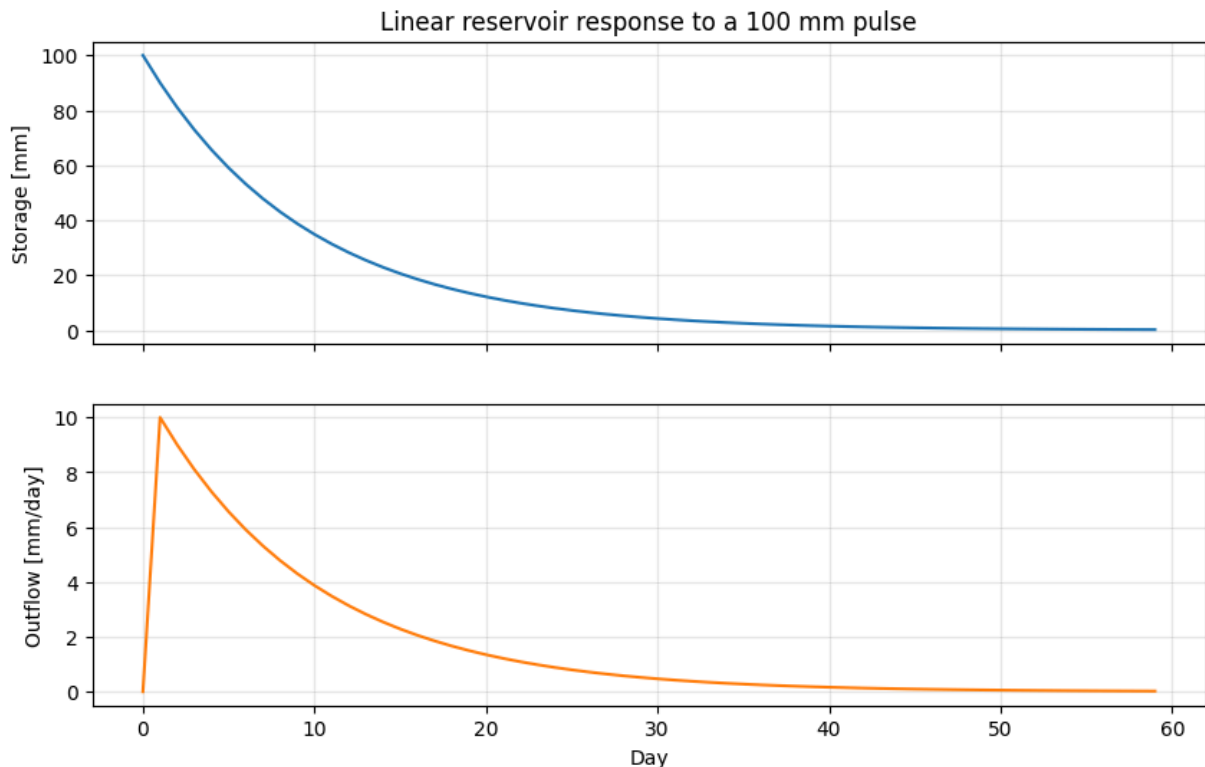
```
peak outflow: 10.0
final storage: 0.19966781110160348
```

```
In [16]: fig, (ax_storage, ax_outflow) = plt.subplots(
        2,
        1,
        figsize=(10, 6),
        sharex=True,
    )

    ax_storage.plot(days, storage_mm, color="tab:blue")
    ax_storage.set_ylabel("Storage [mm]")
    ax_storage.set_title("Linear reservoir response to a 100 mm pulse")
    ax_storage.grid(alpha=0.3)

    ax_outflow.plot(days, outflow_mm, color="tab:orange")
    ax_outflow.set_xlabel("Day")
    ax_outflow.set_ylabel("Outflow [mm/day]")
    ax_outflow.grid(alpha=0.3)

    plt.show()
```



10. Compare Different Residence Times

Now we run the same pulse experiment with three different values of `residence_time_days`.

10. Сравниваем разные времена пребывания

Теперь запускаем тот же импульсный эксперимент с тремя разными значениями `residence_time_days`.

```
In [17]: residence_times_days = [5.0, 10.0, 30.0]

fig, (ax_storage, ax_outflow) = plt.subplots(
    2,
    1,
    figsize=(10, 6),
    sharex=True,
)

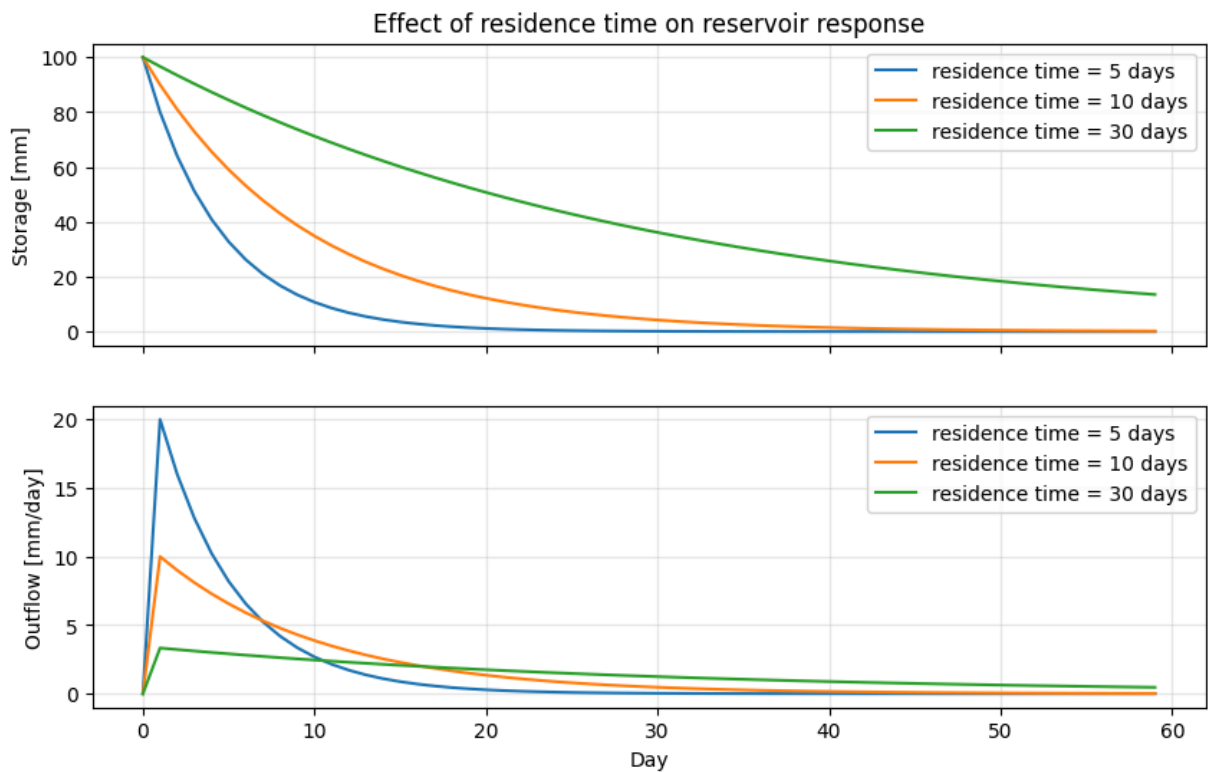
for residence_time_days in residence_times_days:
    storage_mm, outflow_mm = run_linear_reservoir(
        initial_storage_mm=0.0,
        residence_time_days=residence_time_days,
        inflow_series_mm=inflow_series_mm,
    )

    ax_storage.plot(days, storage_mm, label=f"residence time = {residence_time_days}")
    ax_outflow.plot(days, outflow_mm, label=f"residence time = {residence_time_days}")

ax_storage.set_ylabel("Storage [mm]")
ax_storage.set_title("Effect of residence time on reservoir response")
ax_storage.grid(alpha=0.3)
ax_storage.legend()

ax_outflow.set_xlabel("Day")
ax_outflow.set_ylabel("Outflow [mm/day]")
ax_outflow.grid(alpha=0.3)
ax_outflow.legend()

plt.show()
```



Small `residence_time_days` :

- water leaves quickly
- peak outflow is high
- recession is steep

Large `residence_time_days` :

- water leaves slowly
- peak outflow is lower
- recession is longer

The residence time controls the timing of runoff.

Малое `residence_time_days` :

- вода выходит быстро
- пик оттока высокий
- спад резкий

Большое `residence_time_days` :

- вода выходит медленно
- пик оттока ниже
- спад длиннее

Время пребывания управляет временем формирования стока.

11. Optional: Real Alamedin Precipitation

The pulse experiment helped us understand the model.

Now we can drive the same model with real daily precipitation.

11. Дополнительно: реальные осадки Аламедина

Эксперимент с импульсом помог нам понять модель.

Теперь мы можем запустить ту же модель на реальных суточных осадках.

```
In [18]: from pathlib import Path

import pandas as pd

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_ROOT = PROJECT_ROOT / "data" / "02_data"
ALAMEDIN_FOLDER = DATA_ROOT / "15189_Alamedin_River_KGZ"

precip_path = ALAMEDIN_FOLDER / "02_forcing" / "era5_precipitation_daily.csv"

print(precip_path)
print(precip_path.exists())
```

```
/Users/nicolaslazaro/Desktop/work/wat-mod-GIZ/data/02_data/15189_Alamedin_Ri
ver_KGZ/02_forcing/era5_precipitation_daily.csv
True
```

```
In [19]: precip_df = pd.read_csv(precip_path, parse_dates=["date"])
precip_2015_df = precip_df.loc[precip_df["date"].dt.year == 2015].copy()

print("rows:", len(precip_2015_df))
print("annual precipitation:", precip_2015_df["precipitation_mm"].sum())

precip_2015_df.head()
```

```
rows: 365
annual precipitation: 1276.8017
```

Out [19]:

	date	precipitation_mm
12418	2015-01-01	0.0070
12419	2015-01-02	1.7699
12420	2015-01-03	0.5198
12421	2015-01-04	0.1958
12422	2015-01-05	0.9147

```
In [20]: inflow_2015_mm = precip_2015_df["precipitation_mm"].values

storage_2015_mm, simulated_q_2015_mm = run_linear_reservoir(
    initial_storage_mm=0.0,
    residence_time_days=15.0,
    inflow_series_mm=inflow_2015_mm,
)

precip_2015_df["storage_mm"] = storage_2015_mm
precip_2015_df["simulated_q_mm"] = simulated_q_2015_mm
```

```
In [21]: q_path = ALAMEDIN_FOLDER / "03_processed" / "alamedin_cleaned_daily.csv"
q_df = pd.read_csv(q_path, parse_dates=["date"])

q_2015_df = q_df.loc[
    (q_df["date"].dt.year == 2015)
    & (q_df["q_status"] == "observed")
].copy()

print("observed rows:", len(q_2015_df))
print("annual observed Q:", q_2015_df["q_mm_clean"].sum())
print("annual simulated Q:", precip_2015_df["simulated_q_mm"].sum())

q_2015_df.head()
```

```
observed rows: 365
annual observed Q: 540.8842410336915
annual simulated Q: 1264.7956809895272
```


Out [21]:

	date	q_m3s	q_mm	gap_length_days	q_m3s_clean	q_status	q_mm_clean
5479	2015-01-01	1.86	0.388127	0	1.86	observed	0.388127
5480	2015-01-02	1.88	0.392300	0	1.88	observed	0.392300
5481	2015-01-03	1.90	0.396474	0	1.90	observed	0.396474
5482	2015-01-04	1.92	0.400647	0	1.92	observed	0.400647
5483	2015-01-05	1.93	0.402734	0	1.93	observed	0.402734

In [22]:

```

fig, (ax_q, ax_p) = plt.subplots(
    2,
    1,
    figsize=(11, 6),
    sharex=True,
)

ax_q.plot(
    q_2015_df["date"],
    q_2015_df["q_mm_clean"],
    color="tab:blue",
    linewidth=1.2,
    label="Observed Q",
)

ax_q.plot(
    precip_2015_df["date"],
    precip_2015_df["simulated_q_mm"],
    color="tab:orange",
    linewidth=1.2,
    label="Simulated Q",
)

ax_q.set_ylabel("Q [mm/day]")
ax_q.set_title("Alamedin 2015: observed vs simulated streamflow")
ax_q.grid(alpha=0.3)
ax_q.legend()

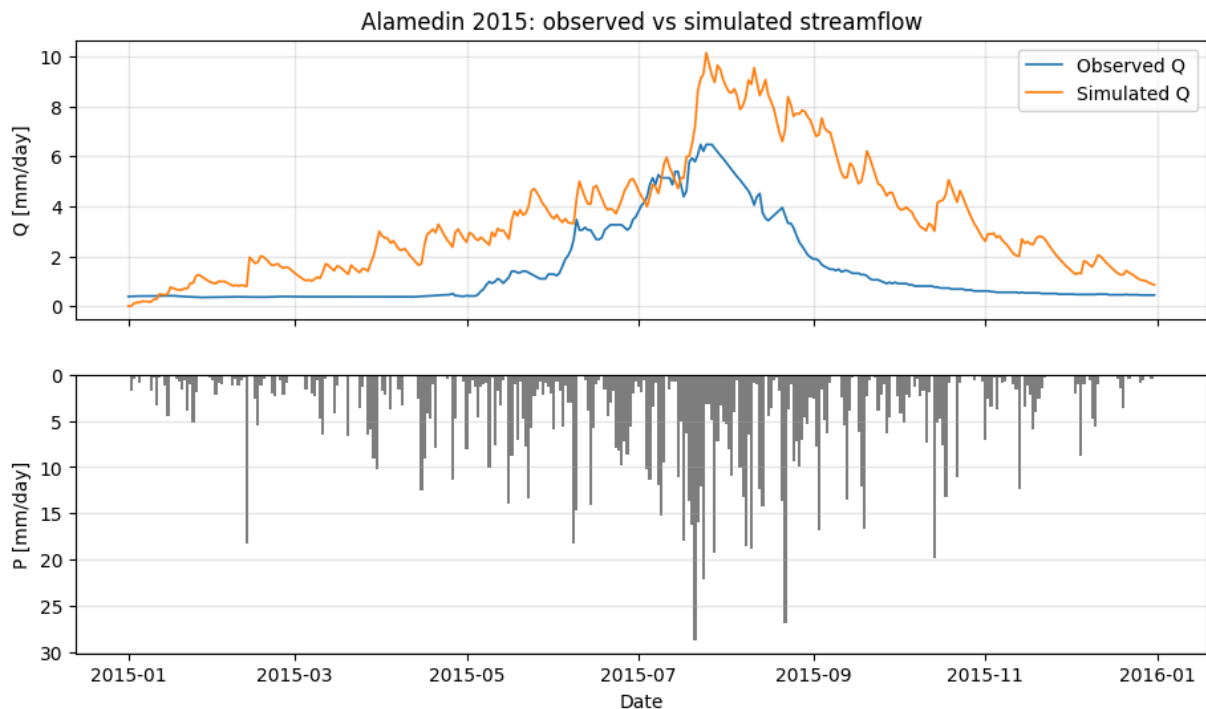
ax_p.bar(
    precip_2015_df["date"],
    precip_2015_df["precipitation_mm"],
    color="tab:gray",
    width=1.0,
)

ax_p.set_ylabel("P [mm/day]")

```

```
ax_p.set_xlabel("Date")
ax_p.grid(axis="y", alpha=0.3)
ax_p.invert_yaxis()

plt.show()
```



This model is useful, but incomplete.

It has no:

- evapotranspiration
- snow accumulation
- snowmelt
- glacier melt
- soil moisture limit
- calibration

But the structure is already the structure of a hydrological model:

forcing → step → storage update → streamflow

Эта модель полезна, но неполна.

В ней нет:

- эвапотранспирации
- накопления снега
- снеготаяния
- таяния ледников
- ограничения почвенной влаги

- калибровки

Но ее структура уже соответствует структуре гидрологической модели:

forcing -> step -> storage update -> streamflow

Final Discussion Questions

1. What does `residence_time_days` control?
2. What happens when `residence_time_days` is small?
3. What happens when `residence_time_days` is large?
4. Why is a `step()` function useful?
5. Why is a `run_linear_reservoir()` function useful?
6. What processes should we add next?

Итоговые вопросы для обсуждения

1. Чем управляет `residence_time_days` ?
2. Что происходит, когда `residence_time_days` маленькое?
3. Что происходит, когда `residence_time_days` большое?
4. Почему функция `step()` полезна?
5. Почему функция `run_linear_reservoir()` полезна?
6. Какие процессы стоит добавить дальше?